

73. Passer à la salle suivante, sans bouton

Ce n'est pas un bouton qui change de salle : c'est le personnage qui sort du cadre.

1. Ce qu'il faut comprendre

La leçon « Passer d'un écran à l'autre » changeait de scène **sur un appui**. Ici, rien de tel : c'est la POSITION du héros qui décide — sortir par la droite fait apparaître la salle suivante, avec le héros collé au bord gauche.

La même règle que `allerA()` s'applique : `ecran(0)`, `effacer()`, redessiner, puis `ecran(1)` — toujours dans cet ordre, pour ne pas voir l'ancienne salle se mélanger à la nouvelle pendant qu'on l'efface.

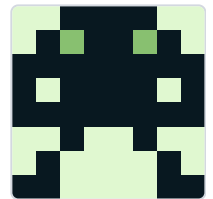
Le héros ne s'arrête jamais à un bord comme dans « Le garder dans l'écran » : ici, dépasser le bord est justement ce qui déclenche le changement. Les deux techniques répondent à deux besoins opposés, avec le même test `x > DROITE_MAX`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1675 octets** de cartouche.

1. UN DESSIN — HEROS

```
Tuile HEROS = {  
    "..####..", ".#-##-#.", "#####", "#.####.#",  
    "#####", "..#..#..", ".#....#.", "##....##",  
};
```



à l'écran

2. UNE CONSTANTE — GAUCHE_MAX

```
const uint8_t GAUCHE_MAX = 8;
```

3. UNE CONSTANTE — DROITE_MAX

```
const uint8_t DROITE_MAX = 152;
```

4. UNE VARIABLE — SALLE

```
uint8_t salle = 0;
```

5. UNE FONCTION — DESSINERSALLE()

```
void dessinerSalle() {
    ecran(0);
    effacer();

    if (salle == 0) {
        texte(6, 1, "SALLE A");
        poser(18, 8, 1);
    } else {
        texte(6, 1, "SALLE B");
        poser(1, 8, 1);
    }

    ecran(1);
}
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t x = 76;
    uint8_t y = 68;

    dessinerSalle();

    while (true) {
        if (bouton(GAUCHE)) x--;
        if (bouton(DROITE)) x++;
        if (bouton(HAUT)) y--;
        if (bouton(BAS)) y++;

        if (x > DROITE_MAX && salle == 0) {
            salle = 1;
            x = GAUCHE_MAX;
            dessinerSalle();
        } else if (x < GAUCHE_MAX && salle == 1) {
            salle = 0;
            x = DROITE_MAX;
            dessinerSalle();
        }

        sprite(0, x, y, HEROS);
        image();
    }

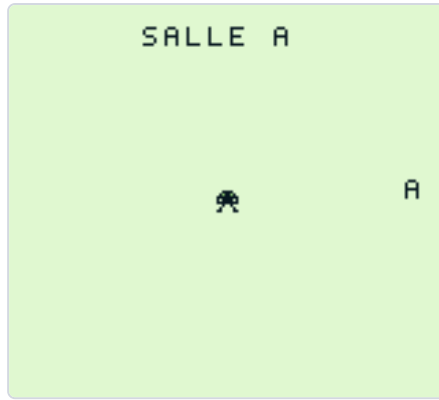
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

« SALLE A », puis « SALLE B » dès que le héros sort par la droite — il réapparaît collé au bord gauche.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on démarre dans la salle A
- ✓ sortir à droite fait passer en salle B
- ✓ le héros réapparaît près du bord gauche ($x = 10$)
- ✓ ressortir à gauche ramène en salle A

Ensuite : **74. Se souvenir d'une partie à l'autre** — niveau 10 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++